

1. API GW / Load Balancer

Виступає єдиною точкою входу для всіх мережових запитів від мобільного додатку системи. Він маршрутизує вхідний трафік до відповідних внутрішніх мікросервісів, використовуючи протокол *REST API* для стандартних операцій (управління профілем, перегляд статистики) та *WebSocket* для безперервного стрімінгу GPS-координат в реальному часі. Технічно цей компонент може бути реалізований як потужний реверс-проксі або спеціалізований шлюз, за допомогою *Nginx* / *Kong API GW* / *AWS API GW*. Основна мотивація для вибору таких інструментів полягає в їхній здатності ефективно розподіляти величезне навантаження між багатьма екземплярами сервісів. Додатково, цей шар бере на себе SSL-термінацію, підтримку довготривалих з'єднань (websockets) та приховує складність внутрішньої мікросервісної архітектури від клієнта.

2. User Profile Service

Відповідає за управління обліковими записами користувачів, включаючи процеси реєстрації, автентифікації та збереження персональної інформації (дані профілю, налаштування тощо). Цей компонент взаємодіє із клієнтським додатком через *API GW* за допомогою стандартних синхронних *REST API* запитів. Технічно сам мікросервіс може бути реалізований як веб-сервер з використанням сучасних технологій, таких як *Spring Boot* / *Go* / *Node.js*. Для фізичного збереження інформації він використовує ізольовану реляційну базу даних *PostgreSQL*. Основна мотивація вибору саме реляційної СУБД полягає у необхідності підтримувати чітку схему даних та гарантувати строгу консистентність для критично важливої інформації 50 млн користувачів.

3. Workout Service

Керує життєвим циклом тренування (старт, пауза, завершення) та відповідає за збереження і відображення детальної інформації про конкретний забіг. Він приймає синхронні *REST API* запити від клієнтів через *API GW*, а також асинхронно отримує фінальні обчислені метрики (дистацію, швидкості) від *Stream Processor*. Для збереження та читання цих загальних метаданих сервіс взаємодіє з ізольованою реляційною базою *PostgreSQL*, тоді як для відмальовки маршруту на карті - читає масив сирих GPS-точок безпосередньо з *Cassandra*. Технічно цей компонент реалізується як масштабований stateless веб-сервіс з використанням продуктивних технологій, таких як *GO* / *Java (Spring Boot)* / *.NET*. Мотивація такого архітектурного вибору полягає в необхідності ефективно оркеструвати дані з кількох різних баз, об'єднуючи їх у єдину цілісну відповідь для користувача.

4. Stream Processor

Критично важливий компонент для обробки даних у реальному часі, який зчитує безперервний потік сирих GPS-координат з черги повідомлень *Kafka*. Він на льоту обчислює диманічні метрики поточного забігу, такі як пройдена дистанція та швидкість, після чого передає ці розрахунки до *Workout Service*. Крім того, процесор відповідає за збереження сирих геолокаційних точок маршруту в базу *Cassandra*, а також паралельно формує і відправляє агреговані події в *ClickHouse* для глобальної статистики. Технічно цей вузол найкраще реалізувати за допомогою спеціалізованих фреймворків потокової обробки, таких як *Apache Flink* / *Spark Streaming*. Основна мотивація вибору таких інструментів - їхня здатність відмовостійко та з мінімальною затримкою обробляти гігантські обсяги подій від мільйонів активних користувачів, паралельно виконуючи складні математичні обчислення в пам'яті для кожного окремого забігу.

5. Statistics Service

Відповідає за формування та надання користувачам агрегованої статистики про їхні заняття спортом за вибраний період часу (сумарний кілометраж, загальний час, середня та максимальна швидкість). Він приймає синхронні *REST API* запити від клієнтів через *API GW* та звертається до аналітичної бази даних *ClickHouse* для швидкого виконання складних запитів по історичних даних. Для зменшення навантаження на базу та миттєвої віддачі часто запитуваних дашбордів, сервіс зберігає та читає попередньо обчислені результати з ін-меморі сховища *Redis Cache*. Технічно цей мікросервіс може бути реалізований як stateless веб-сервер на базі продуктивних платформ, таких як *GO* / *Node.js* / *Python*. Головна мотивація виділення цього функціоналу в окремий сервіс полягає в ізоляції важких аналітичних обчислень від основної транзакційної логіки запису тренувань, що гарантує стабільність системи при високих навантаженнях.

6. Tracking Ingestion Service

Є спеціалізованим компонентом, який відповідає за прийом високочастотного потоку сирих GPS-координат (кожні 500 мс) від мобільних пристроїв під час активного тренування. Він взаємодіє з клієнтами через *API GW* за допомогою постійного двостороннього з'єднання, використовуючи протокол *WebSocket* / *gRPC*. Усі отримані геолокаційні точки сервіс негайно публікує у вигляді подій до брокера повідомлень *Kafka* для їхньої подальшої асинхронної обробки. Технічно цей мікросервіс найкраще реалізувати як асинхронний сервер з використанням технологій, що відмінно справляються з величезною кількістю одночасних мережових з'єднань, таких як *Go* / *Rust* / *Node.js*. Головна мотивація використання саме таких інструментів та протоколів стрімінгу

полягає у необхідності мінімізувати мережеві накладні витрати, дозволяючи системі стабільно і без затримок координати мільйони координат щосекунди.

7. Apache Kafka

Виступає як високопродуктивна розподілена платформа потокової передачі подій та брокер повідомлень, що слугує центральним буфером даних у системі. Цей компонент асинхронно приймає безперервний потік сирих GPS-координат від *Tracking Ingestion Service* через механізм публікації повідомлень у відповідні топіки. Далі ці дані паралельно зчитуються компонентом *Stream Processor* для обчислення метрик у реальному часі та подальшої маршрутизації у постійні сховища. Основна мотивація використання саме цього інструменту полягає в його здатності згладжувати величезні пікові навантаження, виступаючи надійним амортизатором між високонавантаженим входом від десятків мільйонів клієнтів та повільнішими базами даних. Крім того, завдяки збереженню подій на диску та вбудованим механізмам реплікації, *Kafka* гарантує відмовостійкість і те, що жодна координата тренування не буде втрачена навіть при тимчасовому падінні внутрішніх сервісів-споживачів.

8. PostgreSQL User Data

Надійна реляційна база даних, яка використовується для постійного зберігання конфіденційної інформації про користувачів, включаючи їхні облікові дані, профілі, паролі та налаштування додатку. Цей компонент взаємодіє виключно з мікросервісом *User Profile Service* через пряме мережеве з'єднання (TCP/IP) з використанням стандартних баз даних драйверів. Технічно дане сховище реалізується як кластер класичної реляційної СУБД *PostgreSQL*, налаштований з використанням механізмів реплікації для забезпечення високої доступності. Основна мотивація вибору саме реляційної бази даних полягає в необхідності суворого дотримання транзакційних гарантій (ACID), що є критично важливим для безпечних операцій реєстрації та авторизації. Крім того, чітко структурований характер даних профілю ідеально лягає на табличні схеми, що дозволяє підтримувати сувору цілісність та консистентність інформації для 50 млн користувачів.

9. PostgreSQL Run Metadata

Реляційна база даних, призначена для надійного збереження підсумкових результатів та метаданих кожного тренування, таких як загальна тривалість, пройдена дистанція, а також середня і максимальна швидкість. Це сховище не доступне клієнтам напряму і взаємодіє виключно з мікросервісом *Workout Service* через стандартне мережеве з'єднання за допомогою драйверів баз даних (TCP/IP). Технічно компонент реалізований як кластер

класичної СУБД *PostgreSQL*, який фіксує фінальні показники забігу, що були попередньо обчислені в режимі реального часу за допомогою *Stream Processor*. Основна мотивація вибору реляційної бази для цього завдання полягає у необхідності підтримувати строгі зв'язки між користувачами та їхніми тренуваннями (через зовнішні ключі) з гарантією транзакційної цілісності (*ACID*). Крім того, фізичне відокремлення легких метаданих від “важких” масивів сирих GPS-точок дозволяє системі миттєво віддавати користувачеві історію його попередніх забігів без зайвого навантаження на інфраструктуру.

10. Redis Cache

Високопродуктивне in-memo сховище пар “ключ-значення”, яке використовується для тимчасового зберігання попередньо обчисленої та часто запитуваної агрегованої статистики користувачів. Цей компонент безпосередньо взаємодіє з мікросервісом *Statistics Service* через швидкі мережеві з'єднання за допомогою власного протоколу (на базі *TCP*), що гарантує наднизькі затримки на рівні мілісекунд. Коли клієнт відкриває свій профіль, система спочатку намагається швидко отримати готові дані з *Redis*, і лише у разі їх відсутності виконує важкий аналітичний запит до бази *ClickHouse*. Основна мотивація вибору саме цього інструменту полягає у необхідності миттєво віддавати статистичні дашборди, значно покращуючи швидкість реакції додатку та загальний користувацький досвід. Крім того, такий шар кешування виступає надійним щитом, який радикально знижує обчислювальне навантаження на основну аналітичну базу, запобігаючи повторному виконанню ресурсомістких агрегацій.

11. ClickHouse DB

Спеціалізована колоночна аналітична база даних (OLAP), призначена для постійного зберігання та швидкої агрегації колосальних обсягів історичних даних про тренування користувачів за всі 10 років. Вона наповнюється інформацією безперервно та асинхронно, зберігаючи фінальні метрики забігів та події, отримані безпосередньо від компонента *Stream Processor*. З іншого боку, з цією базою взаємодіє мікросервіс *Statistics Service*, який надсилає складні аналітичні SQL-запити через нативні мережеві драйвери для формування статистичних звітів за обраний період часу. Основна мотивація вибору саме колоночного сховища полягає в його архітектурній здатності блискавично виконувати математичні агрегації (підрахунок сумарного кілометражу, середньої швидкості) над мільярдами записів. Впровадження *ClickHouse* дозволяє повністю ізолювати важке аналітичне навантаження від основної транзакційної системи *PostgreSQL*, гарантуючи, що зведені звіти генеруватимуться за частки секунди.

12. Cassandra Raw Geo-Data

Високомасштабована розподілена NoSQL база даних сімейства wide-column, яка використовується для постійного збереження гігантського масиву сирих GPS-координат з кожного тренування. Вона асинхронно наповнюється новими даними, приймаючи безперервний потік геолокаційних точок, що генеруються кожні 500 мілісекунд, від компонента *Stream Processor*. Для відображення історії тренувань з цією базою безпосередньо взаємодіє мікросервіс *Workout Service*, який виконує швидкі запити на читання, щоб відмалювати пройдений маршрут на карті. Головна мотивація вибору саме *Apache Cassandra* полягає в її неперевершеній здатності витримувати колосальні навантаження на запис від мільйонів користувачів без деградації продуктивності.